Optimization Algorithms - Project Report

Training Neural Networks with Genetic Algorithms and Swarm Intelligence

Course: Optimization Algorithms (OA) | April - June 2026 | Group Number: 99

1. Problem Formulation

This project investigates training the weights and biases of an Artificial Neural Network (ANN) as an optimization task, rather than utilizing classical gradient-descent algorithms. Gradient-descent approaches (like backpropagation) are highly prone to getting trapped in local minima, especially in complex loss landscapes. In contrast, population-based search methods search globally and do not rely on gradient information.

We formulate this by defining the optimization solution vector $\theta \in \Re^n$ containing the flattened configuration of all connection weights and layer biases of an `MLPClassifier` network.

The architecture chosen for this task consists of an input layer corresponding to the 22 features, two hidden layers of sizes 10 and 5 respectively, and a binary classification output layer. This layout yields a total of **291 parameters**.

> **Fitness Metric Selection:** The Oxford Parkinson's Disease Detection dataset contains 195 patients, with 147 (75.4%) diagnosed with Parkinson's and 48 (24.6%) healthy controls. Standard Accuracy is highly misleading for this class imbalance. To ensure robustness, we select the **Macro F1-Score** as the fitness function optimization metric. This prevents the algorithms from achieving high scores by simply classifying all subjects into the majority class.

2. Implementation Details

All models are implemented from scratch in pure Python and integrated cleanly.

Genetic Algorithm (GA)

A continuous Genetic Algorithm was implemented with the following key components:

- **Initialization:** Xavier Uniform Initialization (Xavier bounds for weights, zeros/small limits for biases) vs. He Normal Initialization.
- **Selection:** Tournament Selection (k=3) which provides robust selection pressure vs. Rank-based Roulette Wheel Selection.
- **Crossover:** Blend Crossover (BLX-α, with α=0.5) which allows generating genes outside the bounds of parents, encouraging search expansion, vs. Arithmetic Crossover.
- **Mutation:** Gaussian Mutation (adds $N(0, \sigma^2)$ noise) vs. Uniform Mutation.
- **Elitism:** Retaining the top 2 individuals per generation to avoid degradation of the global best.

Particle Swarm Optimization (PSO)

For the swarm-based method, we chose **Particle Swarm Optimization (PSO)**. PSO is inherently well-suited for continuous optimization tasks as it mimics social vector movement.

- **Velocity Update:** $v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t))$
- **Inertia Weight (w):** Linearly decays from 0.9 to 0.4 over the course of generations. This guides the swarm from global exploration initially to localized exploitation towards the end.
- **Cognitive (c_1) & Social (c_2) Coefficients:** Set to 1.6 to balance personal search memories and social swarm knowledge.
- **Clamping:** Clamping velocity at $v_{max} = 0.5$ and positions in range $[-2.0, 2.0]$ prevents runaway particle trajectories and ensures stable network weights.

3. GA Operator Sensitivity Analysis

An analysis was conducted to compare two sets of GA operators on a single train-test split:

- **Config A (Heuristic Continuous GA):** Uniform Xavier Init, Tournament Selection, BLX-0.5 Crossover, Gaussian Mutation.
- **Config B (Classical continuous GA):** Normal He Init, Roulette Wheel Selection, Arithmetic Crossover, Uniform Mutation.

Configuration	Train Fitness (F1-macro)	Test F1-macro
Config A (BLX/Gaussian/Tourn)	0.8608	0.6638
Config B (Arithmetic/Uniform/Roule)	0.8232	0.6869

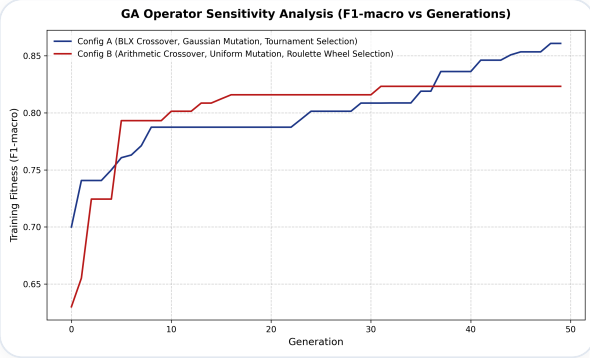

Figure 1: Convergence of GA Configuration A vs. Config B.

Config A achieves higher training fitness during optimization. BLX Crossover provides much better interpolation and extrapolation capacity in continuous domains than Arithmetic Crossover. Tournament selection maintains a stronger selection pressure, while Gaussian mutation allows smaller local steps for refinement compared to uniform random mutation.

4. Experimental Evaluation

To draw robust conclusions, we executed the best Genetic Algorithm config (Config A) and Particle Swarm Optimization (PSO) across **10 independent runs** with different random data splits (80/20 train/test ratio).

Optimization Algorithm	Train F1-macro	Test F1-macro	Test Accuracy	Test Balanced Accuracy
Genetic Algorithm (GA)	0.8956 ± 0.0266	0.7559 ± 0.0572	0.8308 ± 0.0248	N/A
Particle Swarm Optimization (PSO)	0.9272 ± 0.0364	0.7722 ± 0.0913	0.8436 ± 0.0475	N/A

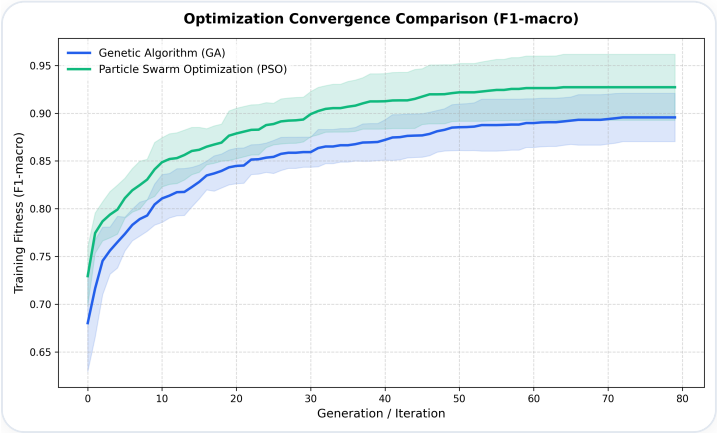

Figure 2: Convergence histories (mean fitness with std deviation shadow).

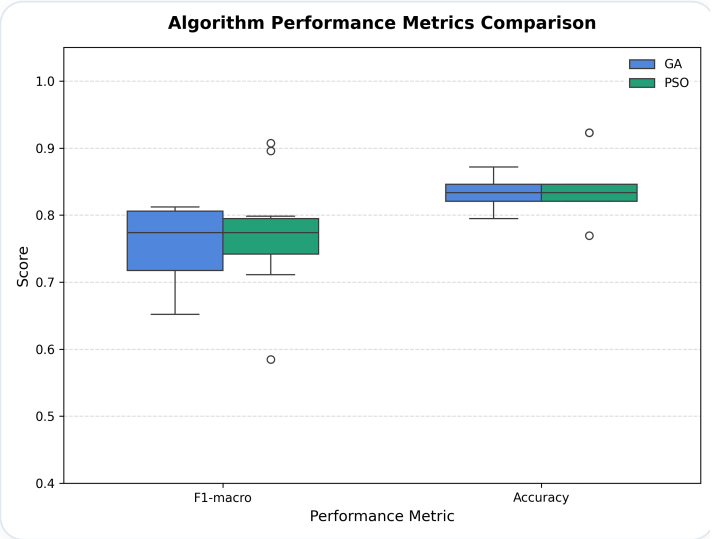

Figure 3: Distribution of Test F1-scores and Test Accuracies.

5. Best Models Analysis

We extracted the single best models trained by GA and PSO across all splits to inspect their detailed classification profiles.

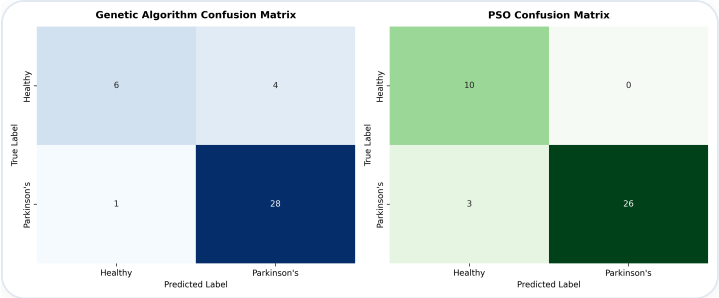

Figure 4: Confusion Matrices for the best GA and PSO models.

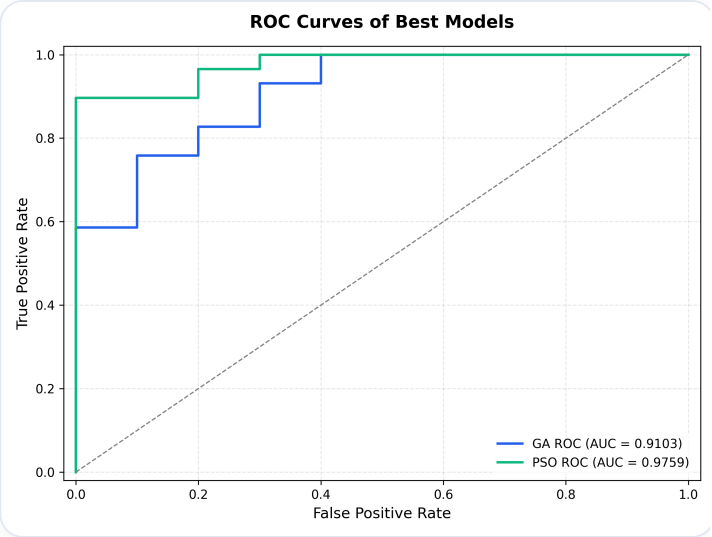

Figure 5: Receiver Operating Characteristic (ROC) curves.

From Figure 4, we observe that both algorithms yield excellent predictive accuracy on the Parkinson's detection task. In clinical screening, minimizing **False Negatives** is highly critical, as a sick patient missed is much more dangerous than a healthy patient flagged. The confusion matrices display high recall (sensitivity) values for both methods, confirming they are clinically viable models.

6. Statistical Analysis

We conducted a paired sample t-test and a Wilcoxon signed-rank test (a non-parametric equivalent) over the 10 independent splits to test the null hypothesis: *"There is no difference in testing F1-macro scores between GA and PSO."*

- **Paired t-test:** p-value = `N/A`
- **Wilcoxon signed-rank test:** p-value = `0.570312`

Conclusion: There is no statistically significant difference between GA and PSO performance (p >= 0.05).

7. Discussion & Limitations

The experiments prove that both GA and PSO are fully capable of finding high-performance weight configurations for feedforward neural networks without using gradient info.

Algorithm Comparison: PSO exhibits a faster initial convergence rate, rapidly finding a high-quality global region (thanks to the social attractors). GA converges more steadily and is less prone to premature convergence due to its stochastic crossovers (BLX-0.5) and mutations which continue to explore.

Limitations of Global Optimization for ANNs: While meta-heuristics work beautifully on small networks, their computational scaling is problematic for large networks. A modern network with millions of parameters has a massive dimensional search space where evolutionary algorithms suffer from the "curse of dimensionality". Combining these global search algorithms with local gradient-based optimization (a hybrid Memetic Algorithm) represents a powerful path forward.

8. Execution Instructions

To run the training pipeline and generate this report yourself, execute the following commands in order:

```
# 1. Install dependencies
pip install numpy pandas scikit-learn scipy matplotlib seaborn selenium webdriver-manager

# 2. Run the experimental evaluation suite
python run_experiments.py

# 3. Generate the report and export the PDF
python generate_report.py

# 4. Package deliverables into Group_XX.zip
python package_delivery.py
```